

```

"""
ARKHEIA-CPS – RTCSA Auto Service
Layer 3: Real-Time Contract Safety Analysis
for AUTO contracts.
Orchestrates OIA-PEM + Perfection Law →
weighted risk score → alerts.
"""

from __future__ import annotations
from dataclasses import dataclass, field
from decimal import Decimal
from datetime import datetime
from typing import List

from sqlalchemy.orm import Session

from app.models.common import Contract,
AnalysisResult, VerticalType
from app.models.auto import
AutoContractDetails
from app.schemas.auto import AutoContractIn
from app.schemas.common import Alert,
AlertSeverity, PEMEvaluation
from app.services.oia_pem_auto import
OIAPEMAutoService
from app.services.perfection_law import
PerfectionLawService
from app.utils.explanations import
generate_auto_explanations
from app.utils.statutes import
get_auto_statutes

```

```

from app.utils.scoring import
compute_weighted_score, score_to_risk_level

# — Dimension weights


---


WEIGHTS = {
    "affordability": 0.25,
    "fee":           0.20,
    "term":          0.20,
    "vehicle_safety": 0.15,
    "enforcement":   0.10,
    "perfection":    0.10,
}

@dataclass
class RTCSAAutoResult:
    overall_risk_score:
        Decimal = Decimal("0")
    risk_level:
        str = "GREEN"
    dimension_scores:
        dict =
        field(default_factory=dict)
    alerts:
        List[Alert] =
        field(default_factory=list)
    pem_evaluations:
        List[PEMEvaluation] =
        field(default_factory=list)
    explanations:
        List[str] =

```

```

field(default_factory=list)
    statutes:
List[dict]          =
field(default_factory=list)
    perfection_days:      int |
None                = None
    illegal_patterns:
List[str]           =
field(default_factory=list)
    predatory_flags:
List[str]           =
field(default_factory=list)

class RTCSAAutoService:

    def __init__(self, db: Session):
        self.db      = db
        self.pem      = OIAPEMAutoService()
        self.perf      = PerfectionLawService()

    def analyze(self, contract: Contract,
details: AutoContractDetails,
                payload: AutoContractIn) ->
RTCSAAutoResult:

        result = RTCSAAutoResult()

        # — Step 1: OIA-PEM evaluation


---


        pem_result =
self.pem.evaluate(payload)
        result.pem_evaluations +=
pem_result.evaluations

```

```

        flags = pem_result.dimension_flags

        # — Step 2: Perfection Law
analysis —————
        perf_result =
self.perf.analyze(payload)
        result.pem_evaluations +=
perf_result.evaluations
        result.alerts +=
perf_result.alerts
        result.perfection_days =
perf_result.perfection_days
        result.illegal_patterns =
perf_result.illegal_patterns
        result.predatory_flags =
perf_result.predatory_flags

        # — Step 3: Dimension scoring
—————

        dim = {}

        dim["affordability"] = (
            85.0 if
flags.get("affordability_fail") else
            35.0 if
flags.get("affordability_warn") else 5.0
        )
        dim["fee"] = (
            75.0 if flags.get("fee_fail")
else
            30.0 if flags.get("fee_warn")
else 5.0
        )
        dim["term"] = 30.0 if

```

```

flags.get("term_warn") else 5.0
        dim["vehicle_safety"] = 10.0    #
stub - integrate VIN/recall API later
        dim["enforcement"] = (
            70.0 if
flags.get("total_cost_fail") or
flags.get("addon_fail") else 15.0
        )
        dim["perfection"] =
float(perf_result.risk_score)

        result.dimension_scores = dim

        # — Step 4: Weighted overall
score —————
        overall =
compute_weighted_score(dim, WEIGHTS)
        result.overall_risk_score =
Decimal(str(overall))
        result.risk_level          =
score_to_risk_level(overall)

        # — Step 5: PEM-sourced alerts
—————

        for ev in pem_result.evaluations:
            if ev.status == "FAIL":
                result.alerts.append(Alert(

code=f"PEM_{ev.pattern_key[:20]}",

severity=AlertSeverity.HIGH,
                message=ev.explanation
or f"{ev.pattern_family} violation
detected.",

```

```

        ))
        elif ev.status == "WARN":
            result.alerts.append(Alert(

code=f"PEM_{ev.pattern_key[:20]}_WARN",

severity=AlertSeverity.MODERATE,
            message=ev.explanation
or f"{ev.pattern_family} deviation
detected.",
            ))

        # Sort alerts: EXTREME → HIGH →
MODERATE → LOW
        severity_order = {"extreme": 0,
"high": 1, "moderate": 2, "low": 3}
        result.alerts.sort(key=lambda a:
severity_order.get(a.severity.value, 4))

        # — Step 6: Plain-language
explanations —————
        result.explanations =
generate_auto_explanations(
            alerts=result.alerts,

perfection_days=result.perfection_days,

illegal_patterns=result.illegal_patterns,
        )

        # — Step 7: Statute references
—————

        result.statutes =
get_auto_statutes(payload.jurisdiction_stat

```

e)

```
        # — Step 8: Persist
        AnalysisResult _____
        self._persist(contract, result)

        return result

    def _persist(self, contract: Contract,
result: RTCSAutoResult):
        dim = result.dimension_scores
        analysis = AnalysisResult(
            contract_id=contract.id,
            vertical=VerticalType.AUTO,

overall_risk_score=result.overall_risk_score,
        risk_level=result.risk_level,

affordability_score=Decimal(str(dim.get("affordability", 0))),

fee_score=Decimal(str(dim.get("fee", 0))),

term_score=Decimal(str(dim.get("term", 0))),

vehicle_safety_score=Decimal(str(dim.get("vehicle_safety", 0))),

enforcement_score=Decimal(str(dim.get("enforcement", 0))),

perfection_risk_score=Decimal(str(dim.get("
```

```

perfection", 0))),
        triggered_alerts_json=[
            {
                "code": a.code,
                "severity": a.severity.value,
                "message": a.message,
                "statute": a.statute,
            }
            for a in result.alerts
        ],
        statutes_json=result.statutes,

        explanations_json=result.explanations,
        pem_evaluations_json=[
            {
                "family":
                e.pattern_family, "key": e.pattern_key,
                "status": e.status,
                "explanation": e.explanation,
                "actual":
                str(e.actual_value) if e.actual_value else
                None,
            }
            for e in
            result.pem_evaluations
        ],
        created_at=datetime.utcnow(),
    )
    contract.analysis_timestamp =
    datetime.utcnow()
    self.db.add(analysis)
    self.db.flush()

```


